

COMBINED TECHNICAL DOCUMENTATION

GraveScanner × GraveVault

The authoritative low-level reference for the two on-chain programs at the core of the GraveYield Protocol: two-phase derelict-pool evaluation, atomic salvage execution, and Merkle-rooted LP claims. This edition is aligned to the shipped implementation, including the Phase 12 adversarial battery that proves the protocol refuses to act when its assumptions fail.

GraveYield Protocol

graveyield.xyz/docs

VERSION 4.0 · OCTOBER 2026 · TDOC-CORE-001

Table of Contents

1. Introduction	1
1.1 What Changed Since Version 3.0	1
1.2 Separation of Concerns	2
1.3 Implementation Status and Conventions	2
2. Shared Data Contracts	3
2.1 EligibilityAnchor PDA	3
2.2 EligibilityCert PDA	4
2.3 LaunchPrice PDA	5
2.4 SalvageReceipt PDA	5
2.5 PoolRegistry and ClaimRecord	6
3. The GraveScanner Program	7
3.1 Instruction Set	7
3.2 The Two-Phase Evaluation Flow	8
3.3 Eligibility Criteria	8
3.4 GraveScanner ProtocolConfig	9
3.5 GraveScanner Error Codes	10
4. The GraveVault Program	11
4.1 Instruction Set	11
4.2 The Atomic Salvage Pipeline	12
4.3 LP Merkle Claims	13
4.4 Dust Sweep	13
4.5 GraveVault ProtocolConfig	14
4.6 GraveVault Error Codes	14
5. Refusal Guarantees - The Phase 12 Adversarial Battery	15
5.1 Method	16
5.2 Headline Refusals	16
5.3 Findings Ledger	17
6. The End-to-End Salvage Lifecycle	18
6.1 Phase 1 - Anchoring	18
6.2 Phase 2 - Certification	18

6.3 Phase 3 - Submission 18

6.4 Phase 4 - Atomic Execution 19

6.5 Phase 5 - LP Claims and Dust Sweep 19

7. PDA Derivation Reference 19

7.1 GraveScanner PDAs 20

7.2 GraveVault PDAs 20

7.3 External Accounts Read 21

8. Document History 21

1. Introduction

This document is the authoritative low-level technical reference for the two on-chain programs that constitute the core of the GraveYield Protocol: **GraveScanner**, the derelict-pool evaluation program, and **GraveVault**, the atomic salvage-execution program. Version 4.0 replaces the Version 3.0 snapshot in its entirety and is aligned to the shipped implementation at commit 27b57ac on GraveYield/graveyield-protocol, which includes the Phase 12 security and economic testnet battery. Every instruction name, account layout, threshold constant, PDA seed, and error code in this edition was verified directly against the Rust sources, the SDK mirrors, and the committed living documentation in the repository at the time of writing.

The document serves three audiences. Protocol engineers use it as the map of what the deployed programs actually enforce, instruction by instruction. Auditors use it as the index into the refusal surface: the sections that follow state not only what the programs do, but what they categorically refuse to do when their assumptions are not satisfied. Integrators and bot operators use it as the contract reference for building against the SDK, whose decoders and error tables mirror the on-chain layouts described here.

1.1 What Changed Since Version 3.0

Version 3.0 (May 2026) documented the protocol as designed. The implementation has since matured through Phases 4 through 12, and several design-time assumptions were deliberately revised on the way to shipped code. Readers familiar with v3.0 should note the following corrections, each of which is detailed in the referenced section:

- **The single `evaluate_pool` instruction became two instructions**, `evaluate_pool_phase_1` and `evaluate_pool_phase_2`, with oracle-attested evidence requirements (Section 3.2).
- **The error-code schemes were replaced wholesale**. GraveScanner now emits 6000-6034 and GraveVault emits 7000-7021; the v3.0 code tables are superseded and most v3.0 names no longer exist (Sections 3.5 and 4.6).
- **Eligibility certificates are reissued, not consumed**. The v3.0 `consumed_at` field does not exist; certs expire by TTL and are reissued in place with a generation counter, while one-shot salvage is enforced structurally by `init-once` vault PDAs (Sections 2.2 and 4.2).
- **NonceRecord and the PoolRegistry bitmap do not exist**. Replay protection is structural: `init-once` PDAs and a `SlotHashes` freshness window (Sections 2.5 and 7).
- **New instructions ship on both programs**: `sweep_stale_anchor` (permissionless rent recovery) and `sweep_dust` (permissionless dust recovery) (Sections 3.1 and 4.4).
- **The fixed 800K CU compute budget figure was removed** as it has no code anchor; compute-unit limit and priority-fee policy are SDK-side per-transaction options bounded by a Charter ceiling (Section 6.3).

- **The Phase 12 adversarial battery is now part of the contract:** fifteen threat classes, 39 dedicated TypeScript cases plus Rust host and fork tests, and a findings ledger feed the audit (Chapter 5).

1.2 Separation of Concerns

GraveScanner and GraveVault communicate exclusively through on-chain PDAs; there is no CPI between them. The integration surface is deliberately narrow: GraveScanner writes the EligibilityAnchor, EligibilityCert, and LaunchPrice PDAs, and GraveVault reads the certificate under a seeds-and-owner constraint that pins it to GraveScanner's program ID. GraveVault writes the PoolRegistry, SalvageReceipt, and ClaimRecord PDAs that the ops layer and the fleet subsequently audit. Neither program trusts instruction arguments for anything the chain can prove on its own, and neither program holds treasury custody of LP positions: the vault directs CPIs as a signer but never takes LP tokens into a program-owned token account.

Responsibility	GraveScanner	GraveVault
Discover and evaluate derelict pools	Yes - primary function	No - trusts the EligibilityCert PDA
Multi-epoch confirmation gate	Yes - EligibilityAnchor + epoch check	No
Verify oracle attestations (ed25519 precompile)	Yes - C1 and C2 evidence	No
Move user funds	Never - read-only evaluation	Yes - inside the atomic salvage transaction
Execute AMM / Jupiter CPIs	No	Yes - redeem and swap
Distribute the 40/40/20 split	No	Yes - salvor, LP pool, protocol
Emit eligibility events	Yes - EligibilityAnchorWritten, EligibilityCertIssued	No
Emit salvage events	No	Yes - PoolSalvaged, SalvageCompleted, LpClaimProcessed, DustSwept

Table 1 - Responsibility matrix between the two programs.

1.3 Implementation Status and Conventions

Both programs are Anchor 0.32.1 workspaces compiled against Solana platform-tools v1.54 and Rust 1.91.1, and both are deployed to devnet with throwaway program keypairs under the KEYS-003 custodial policy (mainnet keys are a pre-mainnet checklist item). The repository is a pnpm workspace whose TypeScript packages mirror the on-chain contract: the SDK (@graveyield/sdk 0.2.0) ships IDL-free instruction builders, byte-locked Merkle tooling, and decoders; the indexer (0.2.0) runs the nine-stage Raydium V4 pipeline; the ops layer (0.1.0) provides services and controlled scenarios; and the adversary package (0.1.0) carries the Phase 12 battery. The snapshotter package produces the sealed LP-holder snapshot artifacts that salvage transactions consume.

Item	Value
Toolchain	Solana CLI 3.0.10, Anchor 0.32.1, Rust 1.91.1 (stable), Node 24 LTS, pnpm 9
GraveScanner program ID (devnet)	5JiCVxES6RYcrFGnFkqKyDmr7fc3EkYaSCbfgJq7zvNF
GraveVault program ID (devnet)	HUyoG5vUmYZJDjdBCxRLLAfm98vEXh63WL3pLARox3v6
External programs referenced	Raydium AMM V4 (675kPX...Mp8), Jupiter v6 (JUP6LkbZbjS1jKKwapdHNy74zcZ3tLUZoi5QNYvTaV4), ed25519 precompile, UNCX Raydium V4 locker (evidence-only)
Test gates at the Phase 12 commit	Scanner 108, Vault lib 33, Vault fork 9, SDK 102, Indexer 29, Ops 47, Adversary 39 - all green

Table 2 - Implementation snapshot at the version 4.0 baseline.

2. Shared Data Contracts

Every program-owned account uses the standard Anchor 8-byte discriminator followed by borsh-serialized fields, with explicit reserved tails that pin account sizes for forward-compatible upgrades. Three PDAs form the scanner-to-vault contract surface and are described first; the vault's own settlement state follows. Field names below are the exact Rust field names, which the SDK decoders mirror byte-for-byte. Where this edition differs from the v3.0 snapshot, the difference is called out explicitly in the surrounding prose rather than silently absorbed.

2.1 EligibilityAnchor PDA

Written once by `evaluate_pool_phase_1` when all six criteria pass. The anchor records the epoch at which the pool first became eligible and freezes the criteria bitmap; Phase 2 later verifies that the live bitmap still equals the anchored one. Seeds: ["eligibility_anchor", amm_program_id, pool_address].

Field	Type	Description
amm_program_id	Pubkey	Owning AMM program of the evaluated pool.
pool_address	Pubkey	The evaluated AMM pool account.
writer	Pubkey	Fee payer that created the anchor; rent is refunded here on sweep.
first_eligible_epoch	u64	Clock epoch at Phase 1. The Phase 2 epoch gate is measured from it.
written_at	i64	Unix timestamp of anchor creation.
invalidated	bool	Set true by multisig <code>invalidate_anchor</code> ; blocks Phase 2 (6017).
criteria_bitmap	u8	All six criteria set at anchor time: 0b00111111 (0x3F).

Field	Type	Description
bump	u8	PDA bump seed.
_reserved	[u8; 64]	Reserved tail; size-pinned.

Table 3 - EligibilityAnchor layout. Renamed from the v3.0 is_valid/anchored_at presentation to the shipped fields.

2.2 EligibilityCert PDA

Written by `evaluate_pool_phase_2` after the epoch gap is confirmed. Seeds: `["eligibility_cert", amm_program_id, pool_address]`. GraveVault reads this PDA with seeds::program pinned to GraveScanner, so a cert minted by any other program fails the vault's account constraints before any state is touched.

The certificate lifecycle changed materially since v3.0. The PDA is created with `init_if_needed` and reissued in place when, and only when, it has expired: `is_expired` is inclusive at `now >= expires_at`, so a cert is live on the half-open window `[issued_at, expires_at)`. Re-running Phase 2 while a cert is still live reverts with `CertStillValid` (6034), which makes two live certs for one pool structurally impossible. Each reissue increments `reissue_generation` and rewrites every field from fresh evidence. There is no `consumed_at` field and no `invalidate_cert` instruction in the shipped program: certificates are retired by TTL, and the one-shot property of salvage is enforced by GraveVault's init-once `PoolRegistry` and `SalvageReceipt` PDAs instead (Section 4.2).

Field	Type	Description
amm_program_id	Pubkey	Owning AMM program.
pool_address	Pubkey	The certified pool.
writer	Pubkey	Fee payer of the issuing Phase 2 transaction.
anchor_epoch	u64	Epoch copied from the EligibilityAnchor at issuance.
cert_epoch	u64	Clock epoch at which Phase 2 was executed.
issued_at	i64	Unix timestamp of issuance.
expires_at	i64	<code>issued_at + cfg.cert_ttl_seconds</code> ; inclusive expiry boundary.
criteria_bitmap	u8	Must equal the anchor bitmap; mismatch reverts 6011.
reissue_generation	u64	1 on first issue; incremented on every in-place reissue.
bump	u8	PDA bump seed.
_reserved	[u8; 56]	Reserved tail; size-pinned.

Table 4 - EligibilityCert layout (spec D10 reissue lifecycle).

2.3 LaunchPrice PDA

The immutable price-collapse baseline for Criterion C2, written once by `record_launch_price`. Seeds: `["launch_price", amm_program_id, pool_address]`. The price is a Q64.64 fixed-point number accepted only inside a 168-byte Ed25519 attestation signed by `ProtocolConfig.launch_price_oracle` and verified by the `ed25519_program` precompile; the instruction's price argument must byte-echo the attested value. Because launch price is a time-invariant historical fact, there is deliberately no `SlotHashes` freshness check; replay is impossible because the PDA is init-once and a second call reverts with 6032-family invariants (zero price 6032, mint mismatch 6033).

Field	Type	Description
<code>amm_program_id</code>	Pubkey	Owning AMM program.
<code>pool_address</code>	Pubkey	The pool whose launch price is recorded.
<code>base_mint</code>	Pubkey	Memecoin side of the recorded pair.
<code>quote_mint</code>	Pubkey	Quote side; must match the live pool before C2 consumes the price.
<code>launch_price_q64x64</code>	u128	Q64.64 launch price; strictly positive (6032 otherwise).
<code>first_swap_slot</code>	u64	Slot of the attested first swap.
<code>first_swap_unix_ts</code>	i64	Timestamp of the attested first swap.
<code>recorded_slot</code>	u64	Slot at which the PDA was written.
<code>recorded_at</code>	i64	Unix timestamp of the write.
<code>bump</code>	u8	PDA bump seed.
<code>_reserved</code>	[u8; 16]	Reserved tail.

Table 5 - LaunchPrice layout (spec D9, ORACLE-001).

2.4 SalvageReceipt PDA

Written by `salvage_pool` at completion and immutable thereafter. Seeds: `["salvage_receipt", pool_address]`. Note a correction to v3.0: the receipt does not carry the Merkle root; the snapshot root lives on `PoolRegistry` (Section 2.5). The receipt instead records the realized 40/40/20 amounts and the dust bookkeeping introduced in Phase 4.

Field	Type	Description
<code>pool_address</code>	Pubkey	The salvaged pool.
<code>salvor</code>	Pubkey	Wallet that executed the salvage.
<code>total_proceeds_lamports</code>	u64	Total base asset recovered after redemption and swap.

Field	Type	Description
salvor_amount_lamports	u64	40% class - transferred immediately.
lp_holder_amount_lamports	u64	40% class - committed to the LP-holder vault.
protocol_amount_lamports	u64	Remainder class - transferred to the protocol treasury.
issued_at_slot	u64	Slot of salvage completion.
issued_at_ts	i64	Unix timestamp of salvage completion.
memecoin_mint	Pubkey	Memecoin that was redeemed; binds sweep_dust to this mint.
dust_memecoin_lamports	u64	Memecoin dust left unsold because it was below the Jupiter dust threshold.
dust_swept_at_ts	i64	Zero until sweep_dust executes.
bump	u8	PDA bump seed.
_reserved	[u8; 32]	Reserved tail.

Table 6 - SalvageReceipt layout; offsets are pinned by unit tests in the program crate.

2.5 PoolRegistry and ClaimRecord

Two further vault accounts complete the settlement picture. PoolRegistry is the per-pool one-shot and accounting anchor: it is initialized inside the first (and only possible) successful salvage, which is what makes a second salvage of the same pool structurally impossible - a deliberate replacement for v3.0's imagined bitmap-plus-nonce design. ClaimRecord is the per-claim double-spend defense for the LP claim path.

Account	Field	Type	Description
PoolRegistry	amm_program_id	Pubkey	Owning AMM program.
PoolRegistry	pool_address	Pubkey	The salvaged pool.
PoolRegistry	salvor	Pubkey	Executor of the salvage.
PoolRegistry	lp_snapshot_merkle_root	[u8; 32]	Root over sha256(holder balance_le) leaves, sorted pairs.
PoolRegistry	lp_total_supply_at_snapshot	u64	LP supply snapshot; must equal live mint supply at salvage (7018).
PoolRegistry	lp_holder_pool_total_lamports	u64	Lamports committed to the LP-holder vault.
PoolRegistry	lp_holder_pool_claimed_lamports	u64	Running total claimed; conservation-checked per claim.
PoolRegistry	salvaged_at_slot / salvaged_at_ts	u64 / i64	Completion slot and timestamp.
PoolRegistry	bump / _reserved	u8 / [u8; 64]	Bump; reserved.

Account	Field	Type	Description
ClaimRecord	pool_address	Pubkey	The salvaged pool.
ClaimRecord	lp_holder	Pubkey	Claimant; init-once, so a second claim reverts at init.
ClaimRecord	amount_lamports	u64	Lamports paid to this holder.
ClaimRecord	lp_balance_at_snapshot	u64	Holder LP balance used in the pro-rata computation.
ClaimRecord	claimed_at_slot / claimed_at_ts	u64 / i64	Claim slot and timestamp.
ClaimRecord	bump / _reserved	u8 / [u8; 32]	Bump; reserved.

Table 7 - PoolRegistry and ClaimRecord layouts. Neither account contains a bitmap field; that v3.0 concept was never shipped.

3. The GraveScanner Program

GraveScanner answers exactly one question - is this pool genuinely derelict? - and answers it with evidence rather than trust. All evaluation inputs that the chain cannot re-derive from pool state (last-swap activity, launch price) arrive as oracle attestations whose signatures are verified by the Solana ed25519 precompile inside the same transaction, and every criterion failure reverts before any PDA is created. The program never holds user funds, never issues CPIs to AMMs or aggregators, and its only state mutations are the four PDAs of Chapter 2 plus config and pause flags.

3.1 Instruction Set

Instruction	Authority	Effect
initialize	multisig	Creates the scanner ProtocolConfig; zero-valued fields fall back to defaults; cert TTL below the 600 s floor reverts 6019.
record_launch_price	oracle-signed	Init-once LaunchPrice PDA from a 168-byte ed25519 attestation (no freshness check by design).
evaluate_pool_phase_1	permissionless + oracle attestation	Writes EligibilityAnchor when all six criteria pass on fresh evidence.
evaluate_pool_phase_2	permissionless + fresh attestation	Writes/reissues EligibilityCert after the epoch gap; bitmap must equal the anchor's.
invalidate_anchor	multisig only	Sets anchor.invalidated; Phase 2 then reverts 6017 until re-anchored.
sweep_stale_anchor	permissionless	Closes an anchor that was never certified once the staleness window elapsed; rent returns to the original writer (6018 if not yet stale).

Instruction	Authority	Effect
update_protocol_config	multisig only	Option-bounded threshold and oracle-key updates; invariant checks revert 6006 / 6019.
emergency_pause	multisig only	Toggles paused; gates evaluate_pool_phase_1/2 and record_launch_price only - sweeps and claims stay live.

Table 8 - The eight GraveScanner instructions. There is no invalidate_cert instruction; certs retire by TTL.

3.2 The Two-Phase Evaluation Flow

Phase 1 - anchor. evaluate_pool_phase_1 requires a 112-byte C1 attestation signed by ProtocolConfig.activity_oracle, delivered as an ed25519_program instruction immediately preceding the scanner instruction. The program checks signature count, offsets, instruction-index binding, oracle identity, and the (amm_program_id, pool_address) binding, then requires the attested slot to resolve in the SlotHashes sysvar - the ~512-slot freshness window that kills replay of old evidence (errors 6024-6031). Locker evidence arrives through remaining_accounts: the UNCX per-pool marker PDA is mandatory whenever it exists on chain (6020/6021), and any supplied TokenLock is re-derived from its own lock_global_id and bound to the pool's (amm_id, lp_mint) pair (6022/6023). On a full pass the program inits the EligibilityAnchor with first_eligible_epoch = clock.epoch and emits EligibilityAnchorWritten. On any failure nothing is written.

Phase 2 - certify. evaluate_pool_phase_2 demands a fresh attestation of the same shape and first gates on the anchor: it must exist (6015), be uninvalidated (6017), and satisfy current_epoch - first_eligible_epoch >= MIN_EPOCH_CONFIRMATION (2; otherwise 6016). The six criteria are then re-evaluated from live state and fresh evidence, and the resulting bitmap must equal the anchor's (6011). On success the cert is created or, if the previous one has expired, reissued in place with reissue_generation incremented; a still-live cert reverts with CertStillValid (6034). The event EligibilityCertIssued carries the generation so downstream consumers can distinguish fresh certificates from reissues.

3.3 Eligibility Criteria

Criteria are evaluated as a fixed sequential conjunction C1 through C6; the first miss reverts. (v3.0 described this as a cheapest-first order; the shipped implementation simply fixes the order below, and every criterion failure surfaces as PoolNotEligible 6001 except where a more specific error exists.) A pool is anchorable or certifiable only when the final bitmap equals ALL_CRITERIA_MASK = 0x3F.

Bit	Criterion	Exact condition	Default threshold
0x01	C1 Inactivity	current_unix_ts - last_swap_unix_ts >= inactivity_seconds, with the attested timestamp validated against Clock (6028) and freshness (6029)	90 days (7,776,000 s)
0x02	C2 Price collapse	drop_bps = floor((launch - current) x 10,000 / launch) >= price_collapse_bps; launch price must be positive and mint-matched (6032/6033)	9,900 bps (99%)

Bit	Criterion	Exact condition	Default threshold
0x04	C3 Residual TVL	current_tvl_lamports >= min_tvl_lamports (inclusive boundary)	0.5 SOL (500,000,000 lamports)
0x08	C4 LP not burned	lp_supply > lp_burn_dust_threshold - strict inequality above the dust threshold	1,000 LP units
0x10	C5 No locked LP	lp_locked_amount == 0, proven via the UNCX marker and TokenLock evidence accounts (6020-6023)	zero locked LP
0x20	C6 Epoch confirmation	Phase 1 sets the bit automatically; Phase 2 requires current_epoch - first_eligible_epoch >= MIN_EPOCH_CONFIRMATION	2 epochs

Table 9 - Criteria C1-C6 in evaluation order. Only C3's and C6's thresholds are configurable-by-governance in spirit; all live in or under ProtocolConfig.

3.4 GraveScanner ProtocolConfig

Field	Type	Default	Notes
inactivity_seconds	u64	7,776,000	C1 threshold (90 days).
price_collapse_bps	u16	9,900	C2 threshold; updates bounded to <= 10,000 (6006).
min_tvl_lamports	u64	500,000,000	C3 threshold.
anchor_staleness_seconds	u64	1,209,600	14-day window used by sweep_stale_anchor, converted to epochs via APPROX_EPOCH_DURATION_SECONDS = 2 days.
lp_burn_dust_threshold	u64	1,000	C4 strict dust floor.
cert_ttl_seconds	i64	3,600	Cert TTL; hard floor 600 s, governance-tunable (6019 below floor).
paused	bool	false	Emergency pause flag.
activity_oracle	Pubkey	= authority at init	Signer of 112-byte C1 attestations.
launch_price_oracle	Pubkey	= authority at init	Signer of 168-byte C2 attestations.
pending_authority / pending_authority_eta	Pubkey / i64	-	Write-only reserved state; the 72-hour timelock is enforced by the Squads governance flow, not on-chain (pre-mainnet TODO).
authority	Pubkey	(init)	Multisig authority.
bump / _reserved	u8 / [u8; 64]	-	Bump; reserved.

Table 10 - Scanner ProtocolConfig. Seeds: ["protocol_config"].

3.5 GraveScanner Error Codes

The table below is the complete shipped scheme, locked by an in-crate unit test that keeps the on-chain codes in sync with the documentation mirror. Discriminants 12-14 (codes 6012-6014) are intentionally reserved for future v4.x additions to the pre-anchor error space; every other code in the contiguous range exists. The v3.0 error tables are superseded: criteria failures collapse into PoolNotEligible (6001) and the attestation family (6024-6031) is new.

Code	Name	Condition
6000	Unauthorized	Caller lacks the multisig authority required.
6001	PoolNotEligible	Pool does not satisfy all six derelict-pool criteria.
6002	LaunchPriceNotFound	LaunchPrice PDA missing for this pool.
6003	UnsupportedAmm	AMM program mismatch or unsupported pool layout.
6004	MathOverflow	Arithmetic overflow during eligibility computation.
6005	InvalidClock	Clock sysvar unavailable or invalid data.
6006	InvariantViolation	ProtocolConfig update violates a locked invariant.
6007	AmmAdapterUnimplemented	AMM adapter registered but parser is a pre-mainnet stub.
6008	LockerAdapterUnimplemented	Retired as a live call site by the UNCX Raydium V4 adapter; kept as a stable code-space slot.
6009	PoolDataParseError	Pool account data did not match the expected layout.
6010	ProtocolPaused	Scanner paused; evaluate_pool_* reverts. Sweeps unaffected.
6011	CriteriaBitmapMismatch	Phase 2 bitmap disagrees with the originating anchor.
6015	AnchorNotFound	Phase 2 attempted without an anchor.
6016	EpochConfirmationPending	Phase 2 before first_eligible_epoch + 2 epochs.
6017	AnchorInvalidated	Anchor voided by multisig; re-evaluate from Phase 1.
6018	AnchorNotStale	sweep_stale_anchor called before the staleness window elapsed.
6019	CertTtlBelowMinimum	Config update or init attempted cert_ttl_seconds below 600 s.
6020	LockerMarkerAccountRequired	UNCX per-pool marker PDA missing while it exists on chain.
6021	LockerLockEvidenceRequired	Marker exists but no TokenLock evidence was supplied.
6022	InvalidLockerAccount	Locker account failed ownership, discriminator, size, or PDA re-derivation checks.
6023	LockerAccountMismatch	TokenLock bound to a different (amm_id, lp_mint) pair.
6024	AttestationMissing	No ed25519 precompile instruction immediately precedes the scanner instruction.

Code	Name	Condition
6025	InvalidAttestationOffsets	Malformed Ed25519SignatureOffsets or wrong instruction-index binding.
6026	AttestationOracleMismatch	Signed key is not the configured oracle.
6027	AttestationBindingMismatch	Attestation binds different values than the instruction params echo.
6028	AttestationTimestampInvalid	Attested timestamp is the zero sentinel or in the future.
6029	AttestationStale	issued_slot no longer resolves in SlotHashes (~512-slot window).
6030	AttestationSlotHashMismatch	Attested slot hash does not match the chain's SlotHashes entry.
6031	AttestationSlotInvalid	issued_slot is zero or in the future.
6032	InvalidLaunchPrice	Attested launch price is zero.
6033	LaunchPriceMintMismatch	Recorded (base_mint, quote_mint) differs from the live pool's mints.
6034	CertStillValid	Phase 2 re-run while the cert is still live; wait for TTL, then reissue.

Table 11 - GraveScanner errors (offset 6000; 6012-6014 reserved).

4. The GraveVault Program

GraveVault converts a valid certificate into a completed, atomically settled salvage - or refuses to run at all. Its entire execution surface is six instructions, of which only salvage_pool moves pool funds. The program holds no LP positions in program-owned token accounts: the Raydium withdraw CPI burns LP tokens in place in the salvor's token account while the vault signs as user_owner through its vault_authority PDA. Everything is one transaction: if any step reverts, no PDA is created, no lamport moves, and the pool is exactly as it was - the property the Phase 12 battery asserts as the atomic no-PDA-residue invariant.

4.1 Instruction Set

Instruction	Authority	Effect
initialize	multisig	Creates the vault ProtocolConfig; enforces share sum == 10,000 bps (7004) and the 20% protocol-share ceiling (7005).
update_protocol_config	multisig only	Option-bounded updates; ceiling and sum invariants re-checked; slippage bounded to <= 10,000 bps.
emergency_pause	multisig only	Toggles emergency_paused; gates salvage_pool only - claims and dust sweeps stay live.
salvage_pool	permissionless + valid cert	The atomic salvage pipeline of Section 4.2.
claim_lp_proceeds	permissionless (Merkle proof)	Pays a holder's pro-rata share against the snapshot root; ClaimRecord init blocks double claims.

Instruction	Authority	Effect
sweep_dust	permissionless	Moves sub-threshold memecoin dust from the vault ATA to the protocol treasury and closes the ATA; one-shot per pool.

Table 12 - The six GraveVault instructions.

4.2 The Atomic Salvage Pipeline

salvage_pool is a single Anchor instruction of roughly nineteen validated steps, presented here in execution order. Steps 1-8 are pure refusals: they touch no vault state, so a malformed attempt costs only fees.

#	Stage	What happens / what is refused
1	Pause gate	emergency_paused reverts 7003. Claims stay live during pause.
2	Certificate gate	Cert must be GraveScanner-owned, unexpired (7002, inclusive boundary), bitmap-complete (0x3F), and bound to the requested pool (7001).
3	Executable checks	Pool and AMM program accounts must be executable and owned by the expected programs (7013).
4	Vault lazy-init	lp_holder_pool_vault and vault_sol_holding_account created on first use via create_account CPI.
5	Orientation proof	coin/pc/lp mints are read at fixed AmmInfo offsets (400/432/464); exactly one side must be WSOL, else UnsupportedBaseToken (7019) - before any CPI.
6	Mint binding	Submitted mint arguments must equal the pool's on-chain bytes (7013).
7	Snapshot equality	lp_mint.supply must equal lp_total_supply_at_snapshot, else InvalidSnapshotData (7018).
8	Remaining accounts split	First 13 = Raydium V4 withdraw bundle; next jupiter_route_accounts_len = the Jupiter route. Miscounting reverts (7013).
9	Route vetting	Nine forbidden custody/state account patterns rejected; the vault's WSOL destination must be present among route accounts (7013).
10	Slippage floor (pre-CPI)	min_quote_output_lamports must clear reserves x (1 - cap/10,000) with cap = min(config max, HARD_MAX 1,000 bps); an override can tighten but never loosen (7007).
11	AMM redemption CPI	Raydium V4 withdraw (tag 4) redeems the full LP balance; tokens burn in place in the salvor's account; non-Raydium AMMs revert 7017.
12	Dust decision	memecoin_received below jupiter_dust_threshold_lamports skips the swap; the dust amount is recorded on the receipt for sweep_dust.
13	Jupiter swap CPI	Jupiter v6 route executed verbatim with the vetted account list; failure or zero output reverts 7016.
14	Output floor (post-swap)	WSOL received must still clear the pre-CPI floor (7007).
15	WSOL unwrap	Vault WSOL closed into vault_sol_holding_account.

#	Stage	What happens / what is refused
16	40/40/20 split	u128 math: salvor and LP shares floored, protocol takes the remainder; sum re-validated against total (7004/7005 on violation).
17	Distribution transfers	Three system transfers signed by vault_authority: salvor, LP-holder vault, protocol treasury.
18	Settlement writes	PoolRegistry (init-once) and SalvageReceipt populated; cert fields left to TTL expiry by design.
19	Events	PoolSalvaged and SalvageCompleted emitted; transaction settles atomically.

Table 13 - The salvage_pool pipeline in execution order.

Refusal before state. Steps 1-8 complete before the first CPI. A pool that fails orientation, mint binding, or snapshot equality is never touched by Raydium or Jupiter, and a failed transaction leaves zero PDA residue - the Phase 12 battery asserts this invariant explicitly (Section 5.2).

4.3 LP Merkle Claims

After a salvage, LP holders claim their share of the LP-holder vault at any time - there is no expiration, and the path stays live during protocol pause. The claimer supplies `lp_balance_at_snapshot` and a Merkle proof; the leaf is `sha256(holder_pubkey || lp_balance_u64_le)` with sorted-pair hashing, byte-locked to the SDK's mirror implementation. The payout is pro-rata over u128: $\text{amount} = \text{lp_holder_pool_total_lamports} \times \text{lp_balance} / \text{lp_total_supply_at_snapshot}$, floored, then conservation-checked against the registry's cumulative claimed counter so rounding can never overpay. The system transfer is signed by the `lp_holder_pool_vault` PDA itself, and the `ClaimRecord` init makes a second claim for the same (pool, holder) structurally impossible (7011 invalid proof, 7011/7012-family duplicates revert). The sealed snapshot artifacts consumed by salvors are produced offline by the snapshotter package and are the same bytes the vault validates on chain. The event `LpClaimProcessed` records each payout for the ops-layer three-way reconciliation.

4.4 Dust Sweep

When a pool's residual memecoin is below the Jupiter dust threshold, the salvage completes without selling it and stamps `dust_memecoin_lamports` on the receipt. `sweep_dust` is the permissionless Phase 4 recovery path: it transfers the vault memecoin ATA's entire balance to the protocol treasury's ATA (destination derived from the treasury PDA, so it cannot be redirected), closes the vault ATA and returns its rent to the caller, and stamps `dust_swept_at_ts` on the receipt. The mint argument is bound to `salvage_receipt.memecoin_mint` (7013), the ATA-balance check refuses empty sweeps (7020), and the receipt stamp makes a second sweep revert (7021). The instruction is pause-independent and emits `DustSwept`.

4.5 GraveVault ProtocolConfig

Field	Type	Default	Notes
lp_holder_share_bps	u16	4,000	40% class.
salvor_share_bps	u16	4,000	40% class.
protocol_share_bps	u16	2,000	Remainder class; can never be raised above the 2,000 bps ceiling (7005).
max_priority_fee_ceiling_lamports	u64	1,000,000,000	Charter rail (1 SOL) read by the SDK's priority-fee policy.
max_slippage_bps	u16	300	Config cap; the pipeline additionally clamps to HARD_MAX_SLIPPAGE_BPS = 1,000.
jupiter_dust_threshold_lamports	u64	666,666	Below this memecoin remainder the swap is skipped (spec D6).
timelock_seconds	i64	259,200	72 h; enforced by the Squads flow - TimelockNotElapsed (7014) is a reserved, never-raised slot.
emergency_paused	bool	false	Gates salvage_pool only.
authority	Pubkey	(init)	Multisig authority; has_one on all admin paths.
bump / _reserved	u8 / [u8; 96]	-	Bump; reserved.

Table 14 - Vault ProtocolConfig. Seeds: ["protocol_config"].

4.6 GraveVault Error Codes

Codes 7000-7021 are the complete shipped set. Four slots are deliberately reserved rather than removed: 7006 is a tripwire that must never be raisable (the LP-holder vault is unsweepable by construction), 7008 is enforced SDK-side against the Charter ceiling rather than on chain, 7012 exists to document that dust below the threshold skips rather than reverts, and 7014 records the governance timelock deferral. The v3.0 names CertExpired, InvalidMerkleProof, AlreadyClaimed, SlippageExceedsMax, and friends are superseded by the names below.

Code	Name	Condition
7000	Unauthorized	Admin path called without the multisig authority.
7001	InvalidEligibilityCert	Cert missing, wrong program, wrong pool binding, or bitmap incomplete.
7002	EligibilityCertExpired	Cert TTL passed (inclusive boundary); re-run scanner Phase 2.
7003	ProtocolPaused	emergency_paused set; salvage_pool unavailable, claims live.

Code	Name	Condition
7004	InvalidShareSplit	Share basis points do not sum to exactly 10,000.
7005	ProtocolShareExceedsCeiling	protocol_share_bps above the 2,000 bps Charter ceiling.
7006	LpHolderPoolUnweepable	Reserved tripwire; never raised - the LP vault is unsweepable by construction.
7007	SlippageExceeded	Floor violated pre-CPI or output floor violated post-swap.
7008	PriorityFeeExceedsCeiling	Reserved; priority fees are enforced by the SDK against the Charter ceiling.
7009	MathOverflow	Arithmetic overflow in proceeds math.
7010	InvalidClaimProof	Merkle proof fails against the registry's snapshot root.
7011	ClaimAlreadyProcessed	ClaimRecord already exists for this (pool, holder).
7012	BelowDustThreshold	Reserved; dust skips the swap leg instead of reverting.
7013	PreflightFailed	Generic pre-flight refusal: account ownership, executable, mint binding, route vetting, or sweep binding.
7014	TimelockNotElapsed	Reserved; timelock enforced by the Squads governance flow.
7015	AmmRedemptionFailed	Raydium withdraw CPI failed (non-Raydium AMMs revert 7017).
7016	JupiterSwapFailed	Jupiter CPI failed or returned zero output.
7017	AmmCpiUnimplemented	CLMM / Orca / PumpSwap redemption requested pre-mainnet.
7018	InvalidSnapshotData	Submitted LP supply does not equal the live mint supply.
7019	UnsupportedBaseToken	Pool orientation is not exactly one WSOL side.
7020	DustNothingToSweep	Vault memecoin ATA is empty.
7021	DustAlreadySwept	Receipt already stamped as swept.

Table 15 - GraveVault errors (offset 7000; 7006/7008/7012/7014 reserved slots).

5. Refusal Guarantees - The Phase 12 Adversarial Battery

Phase 12 shipped a battery whose stated goal is the inverse of a normal test suite: prove that GraveYield refuses to act when its assumptions are not satisfied, because that matters more than proving that it works when everything is normal. The battery lives in the adversary workspace package (@graveyield/adversary 0.1.0) plus Rust host tests inside both program crates and in-process virtual-machine fork tests. Every case maps to exactly one of fifteen threat classes and one of three verdict kinds, and a manifest test proves the mapping is total - no class without a case, no case without a

class.

15 threat classes, roadmap order	39 adversary TS cases (offline)	6 battery layers from Rust to fleet	367 total gate tests green at the commit
-------------------------------------	------------------------------------	--	---

5.1 Method

A case executes in one of six layers: rust-host (pure cargo test of program logic), rust-fork (in-process VM against live program state), ts-sdk, ts-indexer, ts-ops, and fleet (salvor-bots). The verdict vocabulary is deliberately three-valued. **refused** is the happy outcome: the protocol reverted the attack, and a test asserts the exact error. **pinned** records behavior that was examined and consciously accepted - the audit must see it on purpose, not discover it. **finding** marks a gap the battery surfaced, recorded with a stable F-id in docs/ADVERSARY.md for the Phase 13 audit. Case identifiers share one ADV-* space across languages so a single table covers host, fork, and TypeScript suites.

5.2 Headline Refusals

The table below collects the refusal proofs most relevant to the two-program contract of this document. It is a selection, not the full registry; the authoritative class-to-case table is docs/ADVERSARY.md in the repository.

Attack	Threat class	Refusal
Forged LP snapshot supply differs from live mint supply	malicious LP accounts	InvalidSnapshotData 7018; fork test asserts the atomic no-PDA-residue invariant
Certificate TTL configured below the 600 s floor	expired certificates	CertTtlBelowMinimum 6019; positive control passes at exactly 600 s
Re-run Phase 2 while a cert is still live	expired certificates	CertStillValid 6034 - two live certs are structurally impossible
Share configuration violating sum or ceiling	malicious CPI accounts	InvalidShareSplit 7004 / ProtocolShareExceedsCeiling 7005
Unknown AMM program owner presented as a pool	fake dead pools	UnsupportedAmm 6003; registered-but-stub adapters revert AmmAdapterUnimplemented 6007
Maliciously shaped pool account data	malicious LP accounts	PoolDataParseError 6009 (missing accounts, foreign owners, wrong sizes, mint cross-checks)
Clock regression / future-stamped activity evidence	fake timestamps	InvalidClock 6005; attestation family 6028-6031 refuses zero or future timestamps and stale slots
Active pool pushed through the derelict pipeline	active pools	PoolNotEligible 6001 with exact boundary tests at the inactivity and epoch-gap thresholds

Attack	Threat class	Refusal
Zero or manipulated launch price baseline	manipulated prices	InvalidLaunchPrice 6032 / LaunchPriceMintMismatch 6033; collapse boundary pinned exact at 9,900 bps
A single locked lamport of LP	malicious LP accounts	C5 refuses (PoolNotEligible 6001) - the lock check is zero-tolerance
Competing salvor races to the same pool	competing salvors	init-once PoolRegistry; the loser's salvage reverts at init with no state residue
Repeated salvage attempts on a settled pool	repeated salvage attempts	Second salvage reverts at PoolRegistry/SalvageReceipt init; second claim reverts at ClaimRecord init
Dust-scale and extreme-liquidity economics	dust / extreme liquidity	Strict C4 dust boundary; skip-not-revert swap below 666,666 lamports; no upper TVL bound is pinned by design (F2)
Malformed Jupiter routes and hostile route accounts	malformed jupiter routes	Route vetting refuses nine custody/state patterns; missing vault WSOL destination refuses (PreflightFailed 7013)
Token decimals edge cases in price math	token decimals edge cases	Q64.64 price math is decimals-agnostic; orientation derives from mint bytes only (ADV-DC series)
Fork battery: unauthorized signers, pause evasion, CPI authority substitution, fake cert ownership, initialize takeover, claim PDA collisions	cross-class	All revert; nine fork tests green including the three Phase 12 additions

Table 16 - Selected refusal proofs. Full registry: docs/ADVERSARY.md.

5.3 Findings Ledger

The battery also produced findings - gaps that are neither refusals nor silent acceptance but real observations routed to the Phase 13 audit. Each carries a stable identifier so the audit can close them explicitly. The shipped ledger at the version 4.0 baseline:

- **F1 - fee-plan unit mismatch.** The SDK's computeOperationalMaxLamportsPerCu derives a per-CU price from a total budget; documented in the priority-fee policy with the corrected formula pending.
- **F2 - no upper TVL bound.** Whale-scale pools pass C3 by design; the battery pinned this so the economics discussion is explicit rather than accidental.
- **F3 - fleet revalidation order.** The fleet's pre-salvage revalidation does not pre-read PoolRegistry; a competing salvor is discovered at simulation rather than before submission.
- **F5 - expired fleet leases.** An expired fleet lease's assignment is stealable; pinned for the fleet charter discussion.
- **F8 - trailing account bytes.** SDK decoders tolerate trailing bytes after the declared account layout; pinned with an upgrade path.

- **F9 - unknown error codes decode to undefined.** SDK error mapping returns undefined for codes outside the known table; pinned for the audit.

Reading the verdicts. A pinned verdict is not a defect claim - it is the protocol stating its accepted behavior in public so the audit reviews intent rather than discovering surprises. The findings ledger is the only place where the battery claims something is genuinely wrong.

6. The End-to-End Salvage Lifecycle

This chapter walks one pool from discovery to final settlement, naming the exact instructions, evidence, and refusal points at every phase. The lifecycle is intentionally slow at the front (two phases separated by an epoch gate) and instant at the back (one atomic transaction), so that a pool can only become salvageable after standing derelict across multiple epochs and can only be settled in a single all-or-nothing transaction.

6.1 Phase 1 - Anchoring

An operator's tooling discovers a candidate pool through the indexer's nine-stage Raydium V4 pipeline, which pre-filters on reserves, TVL, and scoring before anything is submitted on chain. The operator then assembles `evaluate_pool_phase_1`: an indexer-signed 112-byte activity attestation for C1, the pool and mint accounts, and - where the UNCX marker exists - the locker marker and TokenLock evidence accounts for C5. GraveScanner verifies the attestation against the configured oracle through the `ed25519` precompile, checks SlotHashes freshness, evaluates C1 through C5 (C6 auto-sets at this phase), and on a full pass writes the `EligibilityAnchor` with `first_eligible_epoch` and emits `EligibilityAnchorWritten`. Nothing about the pool is salvageable at this point; the anchor is a claim, not a permission.

6.2 Phase 2 - Certification

At least `MIN_EPOCH_CONFIRMATION` (2) epochs later, the operator re-runs evaluation as `evaluate_pool_phase_2` with a fresh attestation. The epoch gate is checked first (6016 if insufficient, 6015 if no anchor, 6017 if invalidated), then all six criteria are re-evaluated from live state - a pool that reactivated in the meantime now fails and reverts 6001 with no state written. On success the `EligibilityCert` is written or reissued with a TTL of `cert_ttl_seconds` (default one hour, governance floor ten minutes) and `EligibilityCertIssued` is emitted. From this moment the salvage window is real but bounded: when the TTL lapses, the cert is dead and Phase 2 must be re-run, reissuing in place with a new generation.

6.3 Phase 3 - Submission

The salvor fetches a Jupiter quote, estimates profitability against the priority-fee policy, and builds the `salvage_pool` transaction through the SDK. The instruction arguments carry the snapshotter's sealed artifact

data - `lp_snapshot_merkle_root` and `lp_total_supply_at_snapshot` - plus `min_quote_output_lamports` computed from current reserves under the slippage cap, the full Jupiter route bytes, and an optional slippage override that may tighten but never loosen the config cap. A correction to v3.0: there is no fixed 800K CU figure in the code; compute-unit limit is a per-transaction SDK option, and the priority-fee policy applies a 0.25 margin ratio over recent fees while refusing anything above the vault Charter's `max_priority_fee_ceiling_lamports`. Submission goes through a private RPC; the fleet layer adds admission gates and health checks around this step.

6.4 Phase 4 - Atomic Execution

GraveVault executes the nineteen-step pipeline of Section 4.2 inside one transaction. Either every step completes - LP burned in place, memecoin swapped to base, the 40/40/20 split transferred, PoolRegistry and SalvageReceipt written, PoolSalvaged and SalvageCompleted emitted - or the transaction fails and reverts wholesale. There is no partial state: the init-once settlement PDAs are created in the same transaction as the transfers, so a reverted attempt leaves neither a receipt nor a registry row, and the pool can be retried with corrected arguments. The certificate is not consumed on use; it simply expires by TTL, and a second salvage of the same pool is impossible because the registry already exists.

6.5 Phase 5 - LP Claims and Dust Sweep

LP holders claim from the `lp_holder_pool_vault` at any time, with no expiration and no dependency on protocol pause state. Each claim supplies the holder's snapshot balance and Merkle proof; the vault pays the floored pro-rata share, enforces cumulative conservation, initializes the per-(pool, holder) ClaimRecord that makes double claims impossible, and emits LpClaimProcessed. The ops layer's vault observer reconciles claims three ways (sum of claims versus registry counter, registry versus receipt ceiling, and cross-checks) and sweeps recent signatures for failed transactions. If the salvage left sub-threshold memecoin dust, anyone may call `sweep_dust` to return it to the protocol treasury and reclaim the vault ATA rent; the receipt's `dust_swept_at_ts` stamp closes the path after one execution.

7. PDA Derivation Reference

All seeds below are the exact byte strings in the program constants. A correction to v3.0: GraveVault's per-pool PDAs are seeded by `pool_address` alone - `amm_program_id` is stored inside the accounts but is not part of any vault PDA seed, while GraveScanner retains the two-component seed scheme. External accounts are read-only evidence and are listed for completeness because scanner instructions refuse without them when they are required.

7.1 GraveScanner PDAs

PDA	Seeds	Notes
ProtocolConfig	['protocol_config']	Singleton. Multisig-mutable thresholds, oracle keys, pause.
LaunchPrice	['launch_price', 'amm_program_id', 'pool_address']	Init-once Q64.64 launch-price baseline (C2).
EligibilityAnchor	['eligibility_anchor', 'amm_program_id', 'pool_address']	Phase 1 record; voidable by invalidate_anchor; sweepable when stale.
EligibilityCert	['eligibility_cert', 'amm_program_id', 'pool_address']	Phase 2 certificate; reissued in place on TTL expiry.

Table 17 - Scanner PDAs. Seed strings are shown without the b-prefixed Rust literal syntax.

7.2 GraveVault PDAs

PDA	Seeds	Notes
ProtocolConfig	['protocol_config']	Singleton. Shares, slippage, dust threshold, Charter fee ceiling, pause.
PoolRegistry	['pool_address']	Init-once per pool; snapshot root and claim accounting; the structural one-shot salvage defense.
SalvageReceipt	['pool_address']	Immutable receipt of amounts, memecoin mint, and dust bookkeeping.
lp_holder_pool_vault	['lp_holder_pool', 'pool_address']	System-owned, zero-data native-SOL vault; lazy-init; debited only by claim_lp_proceeds (unsweepable, 7006).
ClaimRecord	['claim_record', 'pool_address', 'lp_holder']	Init-once per (pool, holder); the double-claim defense.
protocol_treasury	['protocol_treasury']	Receives the protocol share; owns the dust-sweep destination ATA.
vault_authority	['vault_authority']	Singleton signer PDA for withdraw, swap, unwrap, and distribution transfers.
vault_sol_holding_account	['vault_sol_holding', 'pool_address']	Transient native-SOL holding between WSOL close and distribution.

Table 18 - Vault PDAs. None include amm_program_id in seeds (v3.0 correction).

7.3 External Accounts Read

Account	Owner	Role
UNCX per-pool marker ["global_lp_tracker", amm_id]	UNCX Raydium V4 locker	C5 evidence: its existence makes TokenLock evidence mandatory (6020/6021).
TokenLock ["uncx_locker", lock_global_id.le]	UNCX Raydium V4 locker	146-byte lock record re-derived from its own id; bound to (amm_id, lp_mint); current_locked_amount feeds C5.
AmmInfo (752 bytes)	Raydium AMM V4	Orientation offsets: coin_mint at 400, pc_mint at 432, lp_mint at 464; withdraw instruction tag 4 with 13 remaining accounts.
Jupiter v6 route accounts	Jupiter AGG v6	Forwarded verbatim after structural vetting; vault WSOL destination must be present.
WSOL mint Sol1111...1112	SPL Token	The protocol's base-asset anchor for orientation and floors.

Table 19 - External evidence and execution accounts.

8. Document History

This document carries the TDOC-CORE-001 identifier and follows the repository's terminology contract (salvage, salvor, derelict pool). The frozen PDF snapshots are published for readers and auditors; the living markdown in the repository remains the in-repo source of truth and wins on any disagreement until the next snapshot, per the documentation contract in the repository README. Version 4.0 is the first edition verified against the adversarially-tested implementation rather than the design specification.

Maintenance follows the same sync conventions the repository already enforces for error codes: any change to either program's instruction set, account layout, PDA seeds, or error space must land in the same change that updates this document's successor, and the sync convention treats drift on these surfaces as a bug rather than as documentation debt. A new edition is triggered whenever a phase ships a protocol-visible change - new or renamed instructions, threshold or seed changes, error-space growth, or battery verdict changes - while editorial-only corrections may be batched into the next scheduled revision. The table below records every edition of the combined document to date.

Version	Date	Changes
4.0	October 2026	Full realignment to the shipped implementation at commit 27b57ac (Phases 4-12). Two-instruction evaluation flow with oracle attestations and locker evidence; cert reissue lifecycle replaces consumed_at; error-code schemes replaced with 6000-6034 and 7000-7021 including reserved slots; sweep_stale_anchor and sweep_dust documented; NonceRecord and PoolRegistry bitmap corrections; vault PDA seeds corrected to pool_address-only; new Chapter 5 covering the Phase 12 adversarial battery, its verdict vocabulary, and the findings ledger; lifecycle chapter updated to the SDK priority-fee policy and per-transaction compute-unit options.
3.0	May 2026	Aligned to Whitepaper v4.0. Full terminology reframe: rescue to salvage, rescuer to salvor, dead pool to derelict pool, rescue_pool to salvage_pool, RescueReceipt to SalvageReceipt, RescueCompleted to SalvageCompleted. Introduced multi-epoch confirmation and the two-phase evaluation design as specified at the time.
2.0	April 2026	Aligned to Whitepaper v3.0. Removed GRAVE token, Tombstone NFT, and GRAVE Points references. Merkle-rooted LP holder snapshot. LaunchPrice bound to the first-swap event.
1.0	April 2026	Initial release.

Table 20 - Version history of TDOC-CORE-001.